

DOCUMENTO MAESTRO DEL PROYECTO

Sistema Integrador de Reservas para Community Tours Sian Ka'an

Versión	2.1 - Estado actual del proyecto
Fecha de actualización	20 de agosto de 2026
Estado	Infraestructura inicial validada; preparación para desarrollo incremental
Tecnologías	Electron, HTML/CSS/JavaScript, Node.js + Express, PostgreSQL

1. Descripción general

El Sistema Integrador de Reservaciones tiene como objetivo centralizar y administrar en una sola aplicación de escritorio las reservaciones de Community Tours Sian Ka'an. El sistema busca sustituir la dispersión de información entre plataformas y hojas de cálculo, facilitar la operación diaria y proporcionar trazabilidad de los cambios.

Las fuentes contempladas son FareHarbor, GetYourGuide, WhatsApp Business y reservaciones externas. El sistema no administra pagos; los importes asociados a las reservaciones se conservan como información operativa.

2. Alcance funcional

- Inicio de sesión y control de acceso por roles.
- Dashboard con resumen general de reservaciones.
- Registro, consulta, modificación y cancelación de reservaciones.
- Búsqueda por código, cliente y fecha.
- Historial de reservaciones.
- Bitácora de cambios y trazabilidad.
- Generación del Daily por fecha, vista previa, exportación a PDF e impresión.
- Notificaciones operativas.
- Integraciones futuras con plataformas externas, sujetas a disponibilidad de APIs, permisos y credenciales.

3. Roles del sistema

3.1 Administrador / Reservaciones

- Visualizar, crear, modificar y cancelar reservaciones.
- Consultar historial y bitácora.
- Generar, exportar e imprimir el Daily.
- Consultar notificaciones.

3.2 Usuario de Consulta

- Iniciar sesión.
- Consultar y buscar reservaciones.
- Consultar históricos.
- Sin permisos para modificar información.

La autorización debe comprobarse en el backend; ocultar controles en la interfaz no sustituye la validación de permisos.

4. Arquitectura técnica

Se adoptó una arquitectura por capas con una API REST. La aplicación de escritorio utiliza Electron como contenedor; la interfaz se implementará con HTML5, CSS3 y JavaScript, tomando Google Stitch como referencia visual. El backend utiliza Node.js y Express, y PostgreSQL es el sistema gestor de base de datos.

Electron → Frontend HTML/CSS/JS → HTTP/REST → Node.js + Express → PostgreSQL

El frontend no se conecta directamente a PostgreSQL. Toda operación sobre datos debe pasar por el backend.

5. Tecnologías y entorno

Componente	Tecnología / herramienta	Estado
Base de datos	PostgreSQL + pgAdmin 4	Instalado y validado
Backend	Node.js + Express	Funcionando
Driver PostgreSQL	pg	Instalado y probado
Configuración	dotenv	Instalado
Desktop	Electron	Instalado como dependencia de desarrollo
Frontend	HTML5 + CSS3 + JavaScript	Pendiente de implementación
Diseño UI	Google Stitch	Referencia visual
Editor	Visual Studio Code	Instalado
Versionado	Git	Repositorio local inicializado

6. Modelo de datos

La base de datos de desarrollo se denomina `sian_kaan_reservaciones`. El modelo conserva siete tablas principales y las relaciones establecidas en el DER.

- roles
- usuarios
- tours
- paises
- plataformas
- reservaciones
- bitacora

6.1 Relaciones

Tabla origen	FK	Tabla destino	Relación
usuarios	id_rol	roles	Roles 1:N Usuarios
reservaciones	id_tour	tours	Tours 1:N Reservaciones
reservaciones	id_pais	paises	Países 1:N Reservaciones
reservaciones	id_plataforma	plataformas	Plataformas 1:N Reservaciones
bitacora	id_usuario	usuarios	Usuarios 1:N Bitácora
bitacora	id_reservacion	reservaciones	Reservaciones 1:N Bitácora

6.2 Tabla Reservaciones

El diccionario de datos consolidado define los siguientes campos: `id_reservacion`, `codigo`, `fecha`, `id_tour`, `id_pais`, `id_plataforma`, `nombre_cliente`, `habitacion`, `pax`, `ninos`, `pickup_place`, `pickup_time`, `precio_total`, `deposito`, `saldo`, `tipo_cambio`, `metodo_pago`, `estado`, `fecha_registro` y `ultima_actualizacion`.

Para PostgreSQL se adaptaron únicamente los tipos necesarios: INT a INTEGER, DECIMAL(10,2) a NUMERIC(10,2) y DATETIME a TIMESTAMP. Las claves primarias numéricas utilizan GENERATED ALWAYS AS IDENTITY.

7. Validación de la base de datos

- Se creó la base sian_kaan_reservaciones en PostgreSQL.
- Se crearon y verificaron las siete tablas.
- Se verificaron las seis claves foráneas.
- Se realizaron pruebas de INSERT, SELECT, UPDATE y JOIN.
- Se comprobó la integridad referencial mediante una inserción rechazada por una FK inexistente.
- Se comprobó el registro de acciones en la tabla bitacora.
- Los datos utilizados para estas pruebas son temporales y pueden eliminarse posteriormente.

8. Backend y conexión PostgreSQL

Se inicializó el proyecto Node.js y se instalaron express, pg y dotenv. Electron se instaló como dependencia de desarrollo. La conexión a PostgreSQL está centralizada en backend/config/database.js y utiliza variables de entorno.

El backend dispone de una ruta raíz de prueba que responde con el mensaje 'API del Sistema Integrador de Reservaciones funcionando' y una ruta temporal GET /api/test-db que confirmó la conexión con PostgreSQL.

Navegador → Express → Node.js → pg → PostgreSQL → sian_kaan_reservaciones : VALIDADO

9. Estructura actual del repositorio

```
sistema-integrador-reservaciones/  
■■■ backend/  
■ ■■■ config/database.js  
■ ■■■ controllers/  
■ ■■■ middleware/  
■ ■■■ models/  
■ ■■■ routes/  
■ ■■■ services/  
■ ■■■ server.js  
■■■ database/  
■ ■■■ schema.sql  
■■■ docs/  
■■■ electron/  
■■■ frontend/  
■■■ .env  
■■■ .env.example  
■■■ .gitignore  
■■■ AGENTS.md  
■■■ package.json  
■■■ package-lock.json  
■■■ README.md
```

La carpeta docs ya fue creada para concentrar la documentación del proyecto. El archivo seeds.sql se contempla para una etapa posterior, cuando se definan los datos iniciales definitivos.

10. Git y seguridad del repositorio

- Git fue inicializado localmente.
- Se realizó el primer commit de la estructura funcional inicial.
- El repositorio se encuentra en la rama master.
- El archivo .env está excluido mediante .gitignore y no debe versionarse.
- node_modules, logs y salidas generadas de Electron también deben permanecer fuera del repositorio.

11. Codex

Se definió AGENTS.md como archivo de instrucciones para Codex. Este documento establece el stack obligatorio, arquitectura, estructura, modelo de datos, reglas de seguridad y forma incremental de trabajo.

Codex se utilizará como asistente de implementación, no como sustituto de las decisiones funcionales o arquitectónicas. No debe modificar silenciosamente el modelo de datos, tecnologías, roles, seguridad o integraciones.

12. Decisiones técnicas abiertas

- JWT o sesiones para autenticación.
- Librería para generación del Daily en PDF.
- Estrategia concreta de integración y sincronización con FareHarbor.
- Estrategia concreta de integración y sincronización con GetYourGuide.
- Integración técnica definitiva con WhatsApp Business.
- Estrategia final de empaquetado y distribución de Electron.

13. Estado actual

Elemento	Estado
Arquitectura técnica	Completada
PostgreSQL y pgAdmin	Validado
Modelo y relaciones	Implementados y probados
schema.sql	Preparado
Node.js + Express	Funcionando
Conexión Node.js - PostgreSQL	Validada
Electron	Instalado
Git	Inicializado; primer commit realizado
AGENTS.md	Preparado
README.md	En preparación / actualización
docs/	Carpeta creada
API real de reservaciones	Pendiente
Frontend	Pendiente
Integraciones externas	Pendientes

14. Próximos pasos

- Completar y versionar README.md y la documentación dentro de docs/.
- Definir seeds.sql cuando se aprueben los datos iniciales definitivos.
- Realizar el siguiente commit de documentación/estructura.
- Comenzar el desarrollo incremental con la primera API real: GET /api/reservaciones.
- Separar la implementación del backend en routes, controllers y services.
- Continuar posteriormente con autenticación y roles, frontend, Electron, bitácora, historial, Daily, notificaciones e integraciones.

15. Conclusión

El proyecto ya superó la fase de preparación básica del entorno y validación de infraestructura. PostgreSQL, Node.js, Express y la comunicación entre backend y base de datos se encuentran funcionando. El repositorio Git dispone de un primer punto de control y la estructura está preparada para iniciar la implementación funcional de manera incremental con apoyo de Codex.